

MÁSTER EN INTELIGENCIA ARTIFICIAL
Dev Senior Code

GUÍA DE REPASO COMPLETA
Módulo 1 · Unidad 1
Fundamentos de IA, Ciencia de Datos y Python

¿Para quién es esta guía?

Esta guía está diseñada para estudiantes del Máster que necesitan repasar, reforzar o ponerse al día con los temas de la Unidad 1. Incluye explicaciones conceptuales completas, un caso real desarrollado paso a paso con interpretación de cada resultado, y un ejercicio de aplicación independiente para que compruebes tu comprensión.

No es suficiente con leer el código. Cada sección tiene una explicación del por qué. Si puedes responder '¿por qué se hace esto?' en cada paso, has comprendido el tema.

SECCIÓN 0 | Mapa Conceptual de la Unidad

Antes de entrar al detalle, revisa el panorama completo. Estos son los tres grandes temas de la Unidad 1 y cómo se conectan entre sí:

Tema	Qué cubre y por qué es importante
Tema 1	IA, Machine Learning y Deep Learning: qué son, cómo se diferencian y por qué importa entender la diferencia antes de trabajar con datos.
Tema 2	CRISP-DM: el ciclo de vida estándar de un proyecto de IA. Todo proyecto real sigue este proceso, aunque no siempre de manera lineal.
Tema 3	Python y NumPy: el lenguaje y la librería base para manipular datos numéricos. Sin dominar NumPy, las librerías avanzadas (scikit-learn, TensorFlow) no tienen fundamento.

 Conexión entre los tres temas

Un científico de datos recibe un problema de negocio (Tema 2: CRISP-DM lo guía). Para resolverlo decide si usa ML o DL (Tema 1). Luego abre Python y empieza a manipular los datos con NumPy (Tema 3). Los tres temas son inseparables en la práctica.

SECCIÓN 1 | IA, Machine Learning y Deep Learning

1.1 ¿Qué es la Inteligencia Artificial?

La Inteligencia Artificial (IA) es la disciplina de la informática que busca construir sistemas capaces de realizar tareas que, si las hiciera un humano, requerirían inteligencia: reconocer imágenes, traducir texto, tomar decisiones, predecir comportamientos.

Es importante entender que IA no es sinónimo de robots ni de ciencia ficción. En la práctica, la mayor parte de la IA que usamos hoy es estrecha (narrow AI): sistemas diseñados para hacer una sola tarea muy bien. El corrector ortográfico de tu celular, el sistema de recomendaciones de Netflix, el detector de fraude de tu banco: todos son IA estrecha.

Dato verificado

En 2024, la revista Time reconoció a Rappi como una de las 100 empresas más influyentes del mundo. Una de las razones principales: su uso de IA para predecir demanda y optimizar rutas de entrega en tiempo real. Esto es IA estrecha aplicada a logística.

Fuente: Time 100 Most Influential Companies 2024 / Wikipedia Rappi.

1.2 Machine Learning: aprender de los datos

Machine Learning (ML) es una rama de la IA. La diferencia clave con la programación tradicional está en cómo funciona:

Enfoque	Descripción y ejemplo
Programación tradicional	El humano escribe las reglas explícitamente. Ejemplo: IF precio < 50000 AND descuento > 20% THEN enviar_alerta().
Machine Learning	El humano le da datos y la respuesta correcta. El algoritmo encuentra las reglas por sí solo. Ejemplo: le das 10.000 pedidos históricos con su resultado real, y el modelo aprende qué combinación de variables predice un pedido tardío.

Esta diferencia es fundamental. En ML tú no defines las reglas: tú defines los datos y el algoritmo aprende las reglas desde los patrones.

Por qué esto importa

Imagina que trabajas en Rappi y quieres predecir qué pedidos van a llegar tarde. Con programación tradicional tendrías que escribir cientos de reglas: si llueve Y hora pico Y zona norte... El problema es

que no sabes todas las combinaciones posibles. Con ML, le das el historial de pedidos y el modelo encuentra los patrones automáticamente, incluso los que tú nunca hubieras pensado.

1.3 Deep Learning: aprendizaje con redes neuronales

Deep Learning (DL) es una rama dentro del Machine Learning. Usa arquitecturas llamadas redes neuronales artificiales, inspiradas (vagamente) en cómo funciona el cerebro humano. La palabra 'deep' (profundo) se refiere a que estas redes tienen muchas capas de procesamiento.

Deep Learning sobresale en tareas donde los datos son muy complejos y no estructurados: imágenes, audio, texto largo. No es siempre la mejor opción: si tienes pocos datos o el problema es relativamente simple, ML clásico funciona mejor y es mucho más interpretable.

Tipo	Cuándo usarlo y ejemplo real
Machine Learning clásico	Predecir si un cliente va a cancelar su suscripción (datos tabulares, pocas columnas). Rápido de entrenar, fácil de interpretar.
Deep Learning	Identificar si una foto de un pedido de Rappi muestra el producto correcto o no. Necesita muchas imágenes de entrenamiento. Alta precisión pero más difícil de explicar.

La relación correcta

IA es el campo completo.

Machine Learning es un subconjunto de IA.

Deep Learning es un subconjunto de Machine Learning.

Todo Deep Learning es Machine Learning, pero no todo Machine Learning es Deep Learning.

SECCIÓN 2 | CRISP-DM: El Ciclo de Vida de un Proyecto de IA

CRISP-DM significa Cross-Industry Standard Process for Data Mining. Es el marco de trabajo más usado en la industria para guiar proyectos de ciencia de datos de principio a fin. Fue desarrollado en 1999 por un consorcio de empresas europeas (SPSS, Daimler-Benz, NCR, OHRA) y sigue siendo el estándar de facto en 2024.

La palabra clave es ciclo: CRISP-DM no es una lista lineal de pasos. Es un proceso iterativo. En la práctica, vas a pasar varias veces por las mismas fases porque los datos reales siempre revelan problemas que obligan a volver a etapas anteriores.

2.1 Las seis fases de CRISP-DM

Fase	Descripción detallada
1. Comprensión del negocio	Antes de tocar un solo dato, entiendes el problema: ¿qué quiere lograr el negocio? ¿cuál es el criterio de éxito? En Rappi: ¿queremos reducir tiempos de entrega? ¿Predecir pedidos tardíos? ¿Optimizar rutas? Son preguntas distintas que llevan a modelos distintos.
2. Comprensión de los datos	Exploración inicial: ¿qué datos tenemos? ¿Cuántos registros? ¿Qué columnas? ¿Hay valores faltantes? ¿Los datos tienen sentido? Se usan estadísticas descriptivas y visualizaciones rápidas.
3. Preparación de los datos	La fase más larga (ocupa entre el 60% y el 80% del tiempo real de un proyecto según múltiples estudios de la industria). Limpieza, transformación, creación de nuevas variables, manejo de datos faltantes y atípicos.
4. Modelado	Se seleccionan y entrenan los algoritmos. Aquí es donde entra el ML. Es la fase más visible pero no la más importante en tiempo.
5. Evaluación	Se mide si el modelo realmente resuelve el problema de negocio, no solo si tiene buenas métricas técnicas. Un modelo con 95% de precisión que no detecta el 40% de los fraudes no sirve.
6. Despliegue	El modelo se pone en producción: una API, una aplicación, un sistema automatizado. Sin esta fase, todo el trabajo es solo investigación.

⚠ El error más frecuente en proyectos reales

Saltarse la Fase 1 (Comprensión del negocio) y empezar directamente por los datos. El resultado es construir un modelo técnicamente correcto que no resuelve el problema real. Esto le ocurre con frecuencia a equipos de ciencia de datos que trabajan en silos, sin comunicación constante con el área de negocio.

Fuente del dato sobre el 60-80% del tiempo en preparación de datos: CrowdFlower Data Science Report 2016, citado ampliamente en la literatura de la disciplina.

2.2 CRISP-DM aplicado a Rappi: ejemplo real

Veamos cómo se aplican las seis fases a un problema real de Rappi. El contexto: Rappi quiere predecir qué pedidos tienen alta probabilidad de llegar tarde para alertar proactivamente al cliente y ajustar el tiempo estimado de entrega.

Fase CRISP-DM	Aplicación en Rappi
Fase 1: Negocio	Problema: pedidos que llegan después del tiempo prometido generan solicitudes de reembolso y calificaciones negativas. Meta: modelo que prediga pedidos tardíos con anticipación suficiente para ajustar el tiempo estimado al cliente.
Fase 2: Datos	Datos disponibles: historial de pedidos, coordenadas de restaurantes y usuarios, timestamps de cada evento (confirmación, preparación, recogida, entrega), datos climáticos históricos, datos de tráfico.
Fase 3: Preparación	Muchos pedidos cancelados antes de entregarse (categoría diferente a tardíos). Se crean variables nuevas: distancia restaurante-usuario,

	hora del día, día de la semana. Se excluyen pedidos con datos incompletos.
Fase 4: Modelo	Se prueban tres algoritmos: Regresión Logística (línea base), Random Forest, XGBoost. Se evalúan con validación cruzada.
Fase 5: Evaluación	El modelo con mejor resultado logra buena precisión general, pero falla en pedidos de zonas de alto tráfico en horas pico. Se vuelve a Fase 3 para agregar más variables de tráfico.
Fase 6: Despliegue	El modelo se integra como microservicio: cada vez que se confirma un pedido, la API del modelo predice si será tardío y ajusta el tiempo estimado mostrado al cliente.

💡 Por qué volvieron a la Fase 3

Este es el ciclo de CRISP-DM en acción. La evaluación del modelo reveló que faltaban variables importantes (tráfico por zona y hora). El equipo tuvo que volver a Preparación de Datos para incluirlas. En proyectos reales, esto ocurre siempre. No es un fracaso: es el proceso.

SECCIÓN 3 | Python y NumPy: La Base de Todo

3.1 Por qué Python para IA y ciencia de datos

Python no fue diseñado específicamente para IA ni para ciencia de datos. Es un lenguaje de propósito general creado por Guido van Rossum. Van Rossum comenzó a escribirlo en 1989 y la primera versión pública (0.9.0) se lanzó en febrero de 1991. Sin embargo, se convirtió en el estándar de facto del campo por varias razones que vale la pena entender:

Razón	Por qué importa en la práctica
Legibilidad	Python prioriza que el código sea fácil de leer. En ciencia de datos, donde trabajas con modelos complejos, poder leer el código semanas después es fundamental.
Ecosistema	Tiene las librerías más maduras para datos: NumPy, Pandas, Matplotlib, scikit-learn, TensorFlow, PyTorch. Estas librerías tienen años de desarrollo y comunidades enormes.
Velocidad de prototipado	Permite probar una idea rápidamente. El ciclo entre 'tengo una hipótesis' y 'aquí está el resultado' es muy corto comparado con Java o C++.
Interoperabilidad	Se integra fácilmente con bases de datos, APIs, servicios en la nube y otros lenguajes. Una empresa como Rappi puede conectar su modelo Python con su infraestructura existente.

3.2 NumPy: Arrays y operaciones numéricas

NumPy (Numerical Python) es la librería fundamental para computación numérica en Python. La estructura central de NumPy es el ndarray (n-dimensional array): un arreglo de elementos del mismo tipo, optimizado para operaciones matemáticas.

La diferencia crítica con una lista de Python es el rendimiento. NumPy almacena los datos en bloques contiguos de memoria y realiza operaciones en lote (vectorizadas). Con datasets de tamaño real (miles de filas o más), NumPy puede ser entre 10 y 100 veces más rápido que loops de Python. Para arrays muy pequeños como el de los ejemplos de esta guía, la diferencia es imperceptible: el beneficio real se hace evidente cuando procesas millones de registros.

Por qué NumPy es la base de todo

Pandas usa NumPy internamente para almacenar sus datos.

scikit-learn recibe y devuelve arrays de NumPy.

TensorFlow y PyTorch tienen su propia versión de arrays (tensores) que funcionan de manera similar.

Si entiendes cómo funciona NumPy, entiendes la lógica detrás de todas las librerías de IA.

3.3 Caso desarrollado: Análisis de pedidos de Rappi con NumPy

Vamos a trabajar con datos de estructura similar a la que maneja Rappi. El objetivo: usar NumPy para responder preguntas de negocio concretas sobre tiempos de entrega. Trabajamos con una muestra de 10 pedidos para que puedas seguir los cálculos a mano y verificar cada resultado.

Paso 1: Importar NumPy y crear los arrays de datos

Lo primero es siempre importar la librería. La convención universal es usar el alias 'np'. Esto no es obligatorio técnicamente, pero es tan universal que no usarlo confundiría a cualquier persona que lea tu código.

```
import numpy as np

# Tiempos de entrega prometidos (en minutos)
# Estos son los tiempos que la app mostró al cliente al hacer el pedido
tiempos_prometidos = np.array([30, 25, 40, 35, 28, 45, 32, 38, 27, 42])

# Tiempos de entrega reales (en minutos)
# Tiempo real desde que el cliente confirmó hasta que recibió el pedido
tiempos_reales = np.array([33, 24, 52, 35, 31, 44, 41, 37, 29, 58])

# Zonas de entrega (1=Norte, 2=Sur, 3=Centro, 4=Occidente)
zonas = np.array([1, 2, 3, 1, 2, 4, 3, 1, 2, 4])

print('Datos cargados correctamente.')
print('Número de pedidos:', len(tiempos_prometidos))
```

Interpretación: Paso 1

np.array() convierte una lista de Python en un array de NumPy. La diferencia no se ve aquí, se ve cuando hagamos operaciones: en lugar de usar un loop para calcular la diferencia entre prometido y

real, podemos hacerlo en una sola línea.

Fíjate que los datos ya reflejan una realidad del negocio: pedidos 3, 7 y 10 llegaron bastante tarde (52 vs 40, 41 vs 32, 58 vs 42 minutos). Eso ya es información valiosa antes de cualquier análisis.

Paso 2: Estadísticas descriptivas básicas

Antes de cualquier modelo, siempre describes los datos. Las estadísticas descriptivas te dan una radiografía inicial: ¿cuál es el comportamiento típico? ¿Qué tanto varían los datos? ¿Hay valores extremos?

```
# Promedio de tiempos reales de entrega
promedio_real = np.mean(tiempos_reales)
print(f'Tiempo promedio real: {promedio_real:.1f} minutos')

# Mediana de tiempos reales
mediana_real = np.median(tiempos_reales)
print(f'Mediana tiempo real: {mediana_real:.1f} minutos')

# Desviación estándar
std_real = np.std(tiempos_reales)
print(f'Desviación estándar: {std_real:.1f} minutos')

# Mínimo y máximo
print(f'Tiempo mínimo: {np.min(tiempos_reales)} minutos')
print(f'Tiempo máximo: {np.max(tiempos_reales)} minutos')
```

Resultado esperado:

```
Tiempo promedio real: 38.4 minutos
Mediana tiempo real: 36.0 minutos
Desviación estándar: 10.5 minutos
Tiempo mínimo: 24 minutos
Tiempo máximo: 58 minutos
```

Interpretación: Paso 2

Promedio (38.4 min): El pedido típico tarda 38.4 minutos en llegar. Pero el promedio solo es útil si los datos no tienen valores extremos muy alejados.

Mediana (36 min): La mitad de los pedidos llegan en menos de 36 minutos y la otra mitad en más. La mediana es más robusta ante valores extremos que el promedio.

Que promedio (38.4) sea mayor que mediana (36): indica que hay pedidos con tiempos muy altos que elevan el promedio. El pedido de 58 minutos es un ejemplo claro.

Desviación estándar (10.5 min): Los tiempos se alejan del promedio en promedio 10.5 minutos. Es una variabilidad considerable: la experiencia del cliente es muy inconsistente.

Para el negocio: si Rappi prometió en promedio 34.2 minutos (promedio de tiempos_prometidos) y la realidad es 38.4, hay un déficit promedio de 4.2 minutos. Eso explica calificaciones negativas y solicitudes de reembolso.

Paso 3: Operaciones vectorizadas — calcular el retraso por pedido

Una de las ventajas más importantes de NumPy es la vectorización: puedes aplicar una operación a todos los elementos de un array simultáneamente, sin escribir un loop. Esto es más rápido y más legible.

```
# Diferencia entre tiempo real y prometido (positivo = retraso, negativo = adelanto)
retraso = tiempos_reales - tiempos_prometidos
print('Retraso por pedido (minutos):', retraso)

# ¿Cuántos pedidos llegaron tarde? (retraso > 0)
pedidos_tardios = np.sum(retraso > 0)
print(f'Pedidos tardíos: {pedidos_tardios} de {len(retraso)}')

# Porcentaje de pedidos tardíos
pct_tardios = (pedidos_tardios / len(retraso)) * 100
print(f'Porcentaje de pedidos tardíos: {pct_tardios:.0f}%')

# Retraso promedio solo de los pedidos que SÍ llegaron tarde
retraso_promedio_tardios = np.mean(retraso[retraso > 0])
print(f'Retraso promedio de pedidos tardíos: {retraso_promedio_tardios:.1f} min')
```

Resultado esperado:

```
Retraso por pedido (minutos): [ 3 -1 12  0  3 -1  9 -1  2 16]
Pedidos tardíos: 6 de 10
Porcentaje de pedidos tardíos: 60%
Retraso promedio de pedidos tardíos: 7.5 min
```

** Interpretación: Paso 3**

La línea '`retraso = tiempos_reales - tiempos_prometidos`' resta los dos arrays elemento por elemento sin ningún loop. Esto es la vectorización de NumPy.

`retraso > 0` crea un array booleano `[True, False, True, False, True, False, True, False, True, True]`. El pedido con retraso = 0 (llegó exactamente a tiempo) no cuenta como tardío. `np.sum()` cuenta los True como 1.

`retraso[retraso > 0]` filtra solo los 6 valores positivos: `[3, 12, 3, 9, 2, 16]`. Su promedio es $(3+12+3+9+2+16)/6 = 45/6 = 7.5$ minutos.

Para el negocio: el 60% de los pedidos en esta muestra llegaron tarde, con un retraso promedio de 7.5 minutos. El pedido 10 (retraso de 16 min) y el pedido 3 (retraso de 12 min) son los casos más críticos. Si este patrón se mantiene a escala, con decenas de millones de pedidos mensuales, el impacto en satisfacción del cliente es enorme.

Nota técnica: el pedido 4 tiene retraso = 0 (llegó exactamente a tiempo). No es un error: `0 > 0` es False, por lo tanto no se cuenta como tardío.

Paso 4: Análisis por zona con indexación avanzada

La indexación booleana de NumPy nos permite filtrar datos según una condición. Esto es la base de cualquier análisis segmentado: no todos los pedidos son iguales, y entender las diferencias por segmento es donde está el valor real.

```
# Analizar zona 4 (Occidente) específicamente
mascara_zona4 = zonas == 4
print('Pedidos en zona 4:', np.sum(mascara_zona4))

tiempos_zona4 = tiempos_reales[mascara_zona4]
retrasos_zona4 = retraso[mascara_zona4]

print(f'Tiempo promedio zona 4: {np.mean(tiempos_zona4):.1f} minutos')
```



```
print(f'Retraso promedio zona 4: {np.mean(retrasos_zona4):.1f} minutos')

# Comparar con el promedio general
print(f'Promedio general: {np.mean(tiempos_reales):.1f} minutos')
print(f'Diferencia zona 4 vs general: {np.mean(tiempos_zona4) -
np.mean(tiempos_reales):.1f} minutos')
```

Resultado esperado:

```
Pedidos en zona 4: 2
Tiempo promedio zona 4: 51.5 minutos
Retraso promedio zona 4: 7.5 minutos
Promedio general: 38.4 minutos
Diferencia zona 4 vs general: 13.1 minutos
```

 Interpretación: Paso 4

'zonas == 4' no compara un solo valor: compara cada elemento del array con 4 y devuelve un array de True/False. A esto se le llama máscara booleana.

'tiempos_reales[mascara_zona4]' aplica esa máscara para quedarse solo con los tiempos de la zona 4.

El resultado es revelador: la zona 4 (Occidente) tarda en promedio 13.1 minutos más que el promedio general. Con solo 2 pedidos la muestra es pequeña, pero con datos reales (millones de pedidos) este tipo de análisis por zona es exactamente lo que los equipos de operaciones de Rappi hacen para decidir dónde abrir nuevos dark stores.

Un dark store es un punto de almacenamiento sin acceso al público, diseñado solo para despachar pedidos. Rappi los usa estratégicamente para reducir tiempos en zonas de alta demanda.

Paso 5: Reshaping — preparar datos para Machine Learning

Muchos algoritmos de Machine Learning requieren que los datos tengan una forma específica: una matriz donde cada fila es una observación y cada columna es una variable. NumPy permite reorganizar arrays con `column_stack()`.

```
# Crear una matriz de features (variables de entrada para un modelo futuro)
# Columna 0: tiempo prometido | Columna 1: zona
X = np.column_stack([tiempos_prometidos, zonas])
print('Matriz de features (primeras 5 filas):')
print(X[:5])
print(f'Forma de la matriz: {X.shape}')
print(f'Interpretación: {X.shape[0]} pedidos, {X.shape[1]} variables')

# Variable objetivo: ¿el pedido llegó tarde? (1=sí, 0=no)
y = (retraso > 0).astype(int)
print('Variable objetivo (tardío=1, a tiempo=0):', y)
```

Resultado esperado:

```
Matriz de features (primeras 5 filas):
[[30  1]
 [25  2]
 [40  3]
 [35  1]
 [28  2]]
Forma de la matriz: (10, 2)
Interpretación: 10 pedidos, 2 variables
```

```
Variable objetivo (tardío=1, a tiempo=0): [1 0 1 0 1 0 1 0 1 1]
```

Interpretación: Paso 5

`np.column_stack()` combina arrays 1D en columnas de una matriz 2D. Esto transforma tus datos individuales en la estructura que scikit-learn espera: X con las variables de entrada, y con la variable a predecir.

`.shape` devuelve una tupla (filas, columnas). Es el primer atributo que debes revisar siempre al cargar un dataset nuevo.

`.astype(int)` convierte True/False a 1/0. Los algoritmos de ML trabajan con números, no con booleanos.

Verifica la consistencia: `y = [1,0,1,0,1,0,1,0,1,1]` tiene seis '1', lo que coincide con los 6 pedidos tardíos del Paso 3.

Este array X y este vector y son exactamente el formato que usarás en Unidad 3 y más adelante con scikit-learn. NumPy no es solo una herramienta del Módulo 1: es la base de todo lo que sigue.

SECCIÓN 4 | Cheat Sheet: Términos y Funciones Clave

Término / Función	Definición
IA (Inteligencia Artificial)	Disciplina que busca sistemas capaces de realizar tareas que requieren inteligencia humana. Campo paraguas que contiene ML y DL.
Machine Learning (ML)	Subconjunto de IA donde los sistemas aprenden patrones desde datos sin reglas explícitas programadas.
Deep Learning (DL)	Subconjunto de ML que usa redes neuronales profundas. Destaca en datos no estructurados (imágenes, audio, texto).
CRISP-DM	Cross-Industry Standard Process for Data Mining. Marco de 6 fases para proyectos de ciencia de datos. Es iterativo, no lineal.
<code>np.array()</code>	Crea un array de NumPy desde una lista de Python. Es la estructura de datos fundamental de NumPy.
<code>np.mean()</code>	Calcula el promedio aritmético de un array. Sensible a valores extremos.
<code>np.median()</code>	Calcula la mediana. Más robusta que el promedio ante valores atípicos.
<code>np.std()</code>	Desviación estándar. Mide qué tanto se dispersan los datos alrededor del promedio.
<code>np.min()</code> / <code>np.max()</code>	Valor mínimo y máximo del array.
<code>np.sum()</code>	Suma todos los elementos. Cuando se aplica a un array booleano, cuenta los True.

Vectorización	Aplicar una operación a todos los elementos de un array a la vez. Más rápido que un loop, especialmente con miles de elementos o más.
Máscara booleana	Array de True/False que se usa para filtrar otro array. Se crea con condiciones: <code>array > valor</code> .
<code>array.shape</code>	Atributo que devuelve las dimensiones del array como tupla (filas, columnas).
<code>np.column_stack()</code>	Combina arrays 1D en columnas de una matriz 2D. Útil para construir matrices de features.
<code>.astype(int)</code>	Convierte el tipo de dato del array. True/False → 1/0.
<code>ndarray</code>	N-dimensional array. La estructura de datos central de NumPy.
Indexación booleana	Seleccionar elementos de un array usando una condición como índice: <code>array[array > 5]</code> .

SECCIÓN 5 | Ejercicio Propuesto Autónomo

Este ejercicio aplica los mismos conceptos del caso Rappi, pero en un contexto diferente. El objetivo es que transfieras el conocimiento, no que copies el código anterior.

Contexto: MercadoLibre Colombia — análisis de tiempos de procesamiento

MercadoLibre es la plataforma de e-commerce más grande de América Latina, fundada en 1999 por Marcos Galperin. Opera en 18 países. En Colombia, es uno de los principales marketplaces con millones de transacciones anuales.

Fuente: MercadoLibre Inc. Annual Report 2023 / Investor Relations.

El equipo de operaciones de MercadoLibre Colombia registró los siguientes datos de procesamiento de órdenes para una semana de diciembre (temporada alta):

Tiempos prometidos de despacho (horas): [24, 48, 24, 72, 48, 24, 36, 48, 72, 24, 48, 36]

Tiempos reales de despacho (horas): [26, 45, 31, 68, 52, 23, 38, 47, 89, 27, 44, 41]

Categoría de producto (1=Electrónica, 2=Ropa, 3=Hogar, 4=Libros): [1, 2, 1, 3, 2, 4, 1, 2, 3, 4, 2, 3]

Instrucciones:

Resuelve cada punto en tu Jupyter Notebook. Cada bloque de código debe ir acompañado de una celda Markdown con tu interpretación del resultado en términos del negocio de MercadoLibre.

Punto 1 — Carga y exploración inicial (CRISP-DM: Fase 2)

- Crea los tres arrays con los datos proporcionados.

- Calcula y muestra: promedio, mediana, desviación estándar, mínimo y máximo de los tiempos reales de despacho.
- En tu celda Markdown: ¿qué te dice la diferencia entre promedio y mediana sobre los datos? ¿Hay pedidos atípicos?

Punto 2 — Análisis de cumplimiento (operaciones vectorizadas)

- Calcula el retraso o adelanto en horas para cada orden (real - prometido).
- Determina cuántas órdenes llegaron tarde y el porcentaje que representan.
- Calcula el retraso promedio únicamente de las órdenes que sí llegaron tarde.
- En tu celda Markdown: ¿este nivel de incumplimiento es preocupante para un e-commerce en temporada alta? Argumenta.

Punto 3 — Análisis por categoría (indexación booleana)

- Identifica qué categoría de producto tiene el mayor retraso promedio.
- Compara ese promedio con el promedio general.
- En tu celda Markdown: ¿qué acciones de negocio podría tomar MercadoLibre con este hallazgo?

Punto 4 — Preparación para ML (reshaping)

- Construye una matriz X con dos columnas: tiempo prometido y categoría.
- Construye un vector y que valga 1 si la orden llegó tarde y 0 si llegó a tiempo.
- Muestra la forma (shape) de X y el contenido de y.
- En tu celda Markdown: ¿qué representa cada fila de X? ¿Para qué usarías X e y en el siguiente módulo del máster?

Punto 5 — Reflexión CRISP-DM

- En una celda Markdown (no código), escribe en qué fase de CRISP-DM está este ejercicio completo.
- ¿Qué preguntas quedarían sin responder que corresponderían a la Fase 1 (Comprensión del negocio)?
- ¿Qué harías en la Fase 3 (Preparación de datos) antes de construir un modelo predictivo con estos datos?

✓ Criterio de autoevaluación

Tu análisis está completo si: (1) cada resultado numérico tiene una interpretación en lenguaje de negocio, no solo técnica; (2) puedes explicar por qué usaste cada función de NumPy en lugar de un loop de Python; (3) la reflexión de CRISP-DM conecta explícitamente los pasos del ejercicio con las fases del proceso.

Si puedes responder estas tres preguntas, estás listo para avanzar a la Unidad 2.

Máster en Inteligencia Artificial · Dev Senior Code · Módulo 1 · Unidad 1

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados

[linkedin.com/in/ana-alvarado-instructora-full-stack](https://www.linkedin.com/in/ana-alvarado-instructora-full-stack)